ThinkPad Link: Project Documentation

Two Linux ThinkPads sending messages and files to each other with Python

Repository: https://github.com/bee-0207/thinkpad-link
Date: October 17, 2026
Status: Working. Tested between both ThinkPads on October 16, 2026.

---

1. OVERVIEW

ThinkPad Link connects two laptops over a local network and lets either one send chat messages and files to the other, in both directions at the same time.

The whole program is one Python file, `peer.py`. Both laptops run the same file:

- Laptop A runs it in "listen" mode and waits for a connection.
- Laptop B runs it in "connect" mode and dials laptop A's IP address.
- Once connected, it does not matter which side listened. Both can type messages or send files at any time.

Every file is checked with a SHA-256 checksum when it arrives. If a single byte is damaged, the file is discarded and the sender is told to resend it. Received files are saved in a folder called "received".

2. HARDWARE AND SOFTWARE

The setup used for the working test:

Laptop A (listener)

- Lenovo ThinkPad T15 Gen 1, Ubuntu Linux
- User name: telemetry-hub-1
- Code folder: main-A
- IP address during the test: 172.16.2.39

Laptop B (connector)

- Lenovo ThinkPad T490, Ubuntu Linux
- User name: telemetry-hub-2
- Code folder: `~/Desktop/main-b`

Requirements

- Python 3.8 or newer on both laptops. Check with: `python3 --version`
- Nothing to install with pip. `peer.py` only uses Python's standard library.
- Both laptops on the same network: a phone hotspot or an Ethernet cable. Campus Wi-Fi (eduroam, BU Guest) blocks laptops from talking to each other, so it will not work there.

3. STEP-BY-STEP SETUP

STEP 1: Get the code onto both laptops

Download it from GitHub (the repository is public, no login needed):

```
cd ~
git clone https://github.com/bee-0207/thinkpad-link.git
cd thinkpad-link
```

Any folder name works. In our test the files were in main-A on laptop A and `~/Desktop/main-b` on laptop B. Each folder only needs `peer.py` (and HOW-TO-RUN.txt for reference).

To update just `peer.py` in an existing folder:

```
curl -O https://raw.githubusercontent.com/bee-0207/thinkpad-link/main/peer.py
```

STEP 2: Put both laptops on the same network

Pick one option:

- Phone hotspot (easiest): connect both ThinkPads to the same hotspot.
- Ethernet cable (fastest): see HOW-TO-RUN.txt, Step 1, Option B. It gives laptop A the fixed address 192.168.50.1 and laptop B 192.168.50.2.

STEP 3: Check the firewall on laptop A

```
sudo ufw status
```

If it says "inactive", there is nothing to do. If it says "active":

```
sudo ufw allow 5050/tcp
```

STEP 4: Start laptop A (the listener)

Open a regular terminal (not inside Claude) and run:

```
cd ~/Desktop/main-A
python3 peer.py listen
```

It prints this laptop's IP address and waits. Write down the IP. In our test it was 172.16.2.39.

If the folder is somewhere else, find it with:

```
find ~ -type d -name "main-A" 2>/dev/null
```

STEP 5: Start laptop B (the connector)

Use laptop A's IP address, typed on its own with no brackets around it:

```
cd ~/Desktop/main-b
python3 peer.py connect 172.16.2.39
```

B keeps retrying for 30 seconds, so it is fine to start it a little before A. When both screens say "Connected.", the link is up.

STEP 6: Use it

Type in the `peer.py` window on either laptop:

```
hello there              sends a chat message
/send hello world.txt    sends a file from the current folder
/send /home/telemetry-hub-2/Documents/notes.txt
                         sends a file by its full path
/quit                    disconnects
```

Spaces in file names are fine. Quotes are optional: `/send` "hello world.txt" also works.

What you see after sending a file:

- On the sending laptop: [sent hello world.txt: ...] followed by [peer says hello world.txt arrived intact]
- On the receiving laptop: [got hello world.txt: saved to ..., checksum OK]

The received file goes into the "received" folder inside the folder `peer.py` was run from, for example `~/Desktop/main-A/received/`.

To make a quick test file (in a second terminal, regular bash, so the quotes ARE needed here):

```
echo "hi from the other thinkpad" > ~/Desktop/main-b/"hello world.txt"
```

4. PROBLEMS WE HIT AND HOW WE FIXED THEM

4.1 `peer.py` would crash on Python versions older than 3.12

Problem: One line put double quotes inside a double-quoted f-string:

```
say(f"This laptop's IP address(es): {', '.join(ips) or "unknown, run 'ip a'"}")
```

Python only allows that from version 3.12 on (PEP 701). On Ubuntu 22.04 (Python 3.10) the whole file fails with a SyntaxError before it starts.

Fix: Moved the expression into its own variable. Pushed to GitHub as commit de8c629 ("Fix f-string that only parses on Python 3.12+"):

```
shown = ", ".join(ips) or "unknown, run 'ip a'"
say(f"This laptop's IP address(es): {shown}")
```

4.2 "cd: too many arguments"

Problem: Typed cd ~/ main-A with a space after the slash. Bash read it as two separate arguments.

Fix: The path must be one piece with no spaces, and there must be a space between cd and the path:

```
cd ~/main-A
```

4.3 "cd~/main-A: No such file or directory" and "Command 'cd~' not found"

Problem: No space between cd and the path, so bash looked for a command literally called "cd~".

Fix: Always type cd, then a space, then the path.

4.4 Wrong home folder name

Problem: Typed /home/telemetry-hub1-ThinkPad/Desktop/main-A. The home folder is named after the user (telemetry-hub-1), not the computer (telemetry-hub-1-ThinkPad-T15-Gen-1).

Fix: Use ~, which always means "my home folder":

```
cd ~/Desktop/main-A
```

Tip: Type part of a name, like cd ~/Desktop/ma, and press Tab. The terminal finishes it for you, so typos can't creep in.

4.5 find printed nothing

Problem: The command was typed as -type d name "main-A". The "find ~" at the start and the dash before "name" were missing.

Fix:

```
find ~ -type d -name "main-A" 2>/dev/null
```

4.6 "syntax error near unexpected token `newline'"

Problem: Typed python3 peer.py connect <172.16.2.39>. The angle brackets in the instructions were placeholders. Bash treats < and > as redirection symbols.

Fix: Type the IP address on its own:

```
python3 peer.py connect 172.16.2.39
```

4.7 Why Claude can't run peer.py for you

peer.py is interactive. It waits for you to type messages. Commands that Claude runs are not attached to your keyboard, and they have a time limit (2 minutes by default, 10 at most). Claude can start peer.py, but nobody could type into it. Run the final listen/connect commands yourself in a regular terminal, or ask Claude to open a new terminal window that runs it:

```
gnome-terminal -- python3 peer.py listen
```

5. TROUBLESHOOTING

"Could not reach ... timed out" (laptop B keeps retrying)

- Wrong IP. Run `peer.py listen` on A again and copy the IP it prints.
- The laptops are not on the same network, or they are on campus Wi-Fi.
- Firewall on laptop A. Redo Step 3.

Test the network from laptop B:

```
ping -c 3 172.16.2.39
```

No replies means the laptops can't see each other on the network.

"Connection refused"

- The network works, but laptop A isn't running `peer.py listen` yet, or the two sides are using different ports.

"No route to host"

- Usually a firewall. Redo Step 3.

SyntaxError mentioning an f-string

- That laptop has the old `peer.py`. Replace it with the fixed one:

```
curl -O https://raw.githubusercontent.com/bee-0207/thinkpad-link/main/peer.py
```

"checksum FAILED"

- The file was damaged in transit. Send it again. If it keeps happening, try a different cable or network.

"connection closed"

- The other side quit, or the network dropped. Press Enter, then start listen on A and connect on B again. Any half-received file is deleted automatically.

Port 5050 already in use

- Pick another port, the same number on both sides:

```
python3 peer.py listen --port 6000
python3 peer.py connect 172.16.2.39 --port 6000
```

6. HOW IT WORKS

- One TCP connection. TCP is full duplex: data flows both ways at once, like a phone call rather than a walkie-talkie.
- Each laptop runs two threads: one only reads incoming data, one only sends. That is why one laptop can receive a file while sending one.
- Everything sent is wrapped in a small "frame": a 4-byte length, then a JSON header saying what it is (message, file, file end, or receipt). File headers are followed by the raw file bytes. This is how the other side knows where one item ends and the next begins.
- Files are written to "name.part" first and renamed to the real name only after the SHA-256 checksum matches, so a broken file never looks complete.
- Incoming file names are stripped of folder parts, so the other laptop can only write inside the "received" folder.
- Nothing is encrypted and there is no password. That is fine on a cable or your own hotspot, but don't run it on a public network.

7. QUICK REFERENCE

Laptop A:

```
cd ~/Desktop/main-A
python3 peer.py listen
```

Laptop B:

```
cd ~/Desktop/main-b
python3 peer.py connect 172.16.2.39
```

While connected:

```
any text           chat message
/send <file>       send a file
/quit              disconnect
```

Received files: the "received" folder next to `peer.py`.

Laptop A's IP can change each time it joins a network. Always use the IP that `peer.py listen` prints, not 172.16.2.39 from memory.